JSON Schema

Validate JSON contracts and apply declared defaults

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL json_schema FROM query_farm;  LOAD json_schema FROM query_farm;
```

01 At a glance

Check JSON documents against a schema inside SQL, report exactly what failed, and fill missing properties from schema defaults. Drafts 4 through 2020-12 are selected by $schema; a schema without one uses 2020-12.

Schema + JSON → Validate → Patch or enrich

02 Core SQL surface

`json_schema_is_valid(schema, data)`

Return TRUE or FALSE; invalid data never raises an error.

`json_schema_errors(schema, data)`

List path, keyword and message per failure; [] when valid.

`json_schema_validate(schema, data)`

Return TRUE, or raise an error naming up to five failures.

`json_schema_validate_schema(schema)`

Check that the schema itself is accepted before relying on it.

`json_schema_patch(schema, data)`

Return JSON Patch operations for missing defaults.

`json_schema_update(schema, data)`

Return the document with defaults applied.

`"$schema": "…/draft-07/schema#"`

Choose draft 4, 6, 7, 2019-09 or 2020-12; unknown values are errors.

03 Keep in mind

Make validation policy explicit

Filter with json_schema_is_valid and report with json_schema_errors; use json_schema_validate where nonconforming input should abort the query. Lint the schema first, so a bad contract cannot quietly mark every row invalid.

04 Flag valid and invalid rows

```
SELECT payload, json_schema_is_valid(
  '{"type":"integer"}', payload
) AS valid
FROM (VALUES ('5'::JSON), ('"five"'::JSON))
  t(payload);
-- 5: true; "five": false
```

No TRY wrapper is needed: a violating row returns false.

05 Explain each failure

```
SELECT unnest(json_schema_errors(
  '{"properties":{"age":{"minimum":0}}}',
  '{"age":-1}'
), recursive := true);
-- /age | minimum | Minimum value is 0 …
```

One row per failing keyword, located by JSON Pointer.

06 Fill a missing property

```
SET VARIABLE schema = '{
  "type":"object",
  "properties":{
    "retries":{"type":"integer","default":3}
  }
}';
SELECT json_schema_validate_schema(
  getvariable('schema'));
SELECT json_schema_update(
  getvariable('schema'), '{}');
-- {"retries":3}
SELECT json_schema_patch(
  getvariable('schema'), '{}');
```

The patch contains an add operation at /retries with value 3.

07 In practice

Drafts, formats and patterns

Declare draft-07 for array-form tuple items. format is validated, and pattern uses RE2, so lookaround and backreferences are schema errors. Remote $ref targets are not loaded. Defaults apply only through patch and update.

Full docs & examples ↗ Source ↗